

у меня есть qt 4.8.6 с редактором форм UI на C++. Я придумала слева окна сделать TabWidget, который будет прикреплен и изменяться будет по изменению размера окна. То есть он будет в своего рода компоновке с формой. Но не знаю как это сделать. Как его привязать. У него ведь нет Layout-ов

Thought for 1 second

User wants to make a TabWidget on the left side of window that resizes with window. They're using Qt 4.8.6 Designer. Need to explain layouts.

Как привязать QTabWidget к окну через компоновку

У самого QTabWidget действительно нет своего layout — layout'ы есть у контейнера-родителя (у QMainWindow, QWidget, QDialog и т.п.). Именно родительский layout и управляет тем, как дочерние виджеты (в т.ч. QTabWidget) растягиваются при изменении размера окна.

Способ 1: через Qt Designer (рекомендую)

- Создайте главное окно (QMainWindow или QWidget) — это корневой виджет формы.
- Перетащите на форму все нужные элементы, например:
 - QTabWidget (слева)
 - справа — что угодно (например, QTextEdit, QLabel и т.д.)
- Выделите корневой виджет формы. Важно: кликните по пустому месту формы, а не по виджету. В дереве объектов (Object Inspector) должен подсветиться сам MainWindow / Form.
- Нажмите кнопку «Разложить по горизонтали» (Lay Out Horizontally) — иконка с тремя горизонтальными полосками на верхней панели, либо меню Form → Lay Out Horizontally. Горячая клавиша Ctrl+I (в Qt Designer) или Ctrl+L в некоторых сборках.

Появится красная рамка — это значит, что у формы теперь есть QHBoxLayout, и все дочерние виджеты в него попали.
- Теперь при растягивании окна QTabWidget и его сосед будут делить пространство.

Как настроить пропорции

У каждого виджета в layout есть свойство:

- sizePolicy → Horizontal Policy:
 - Expanding — виджет тянется
 - Preferred / Fixed — остаётся более-менее постоянным
- minimumSize / maxSize — жёсткие границы.
- layoutStretch у самого layout'a — задаёт пропорции. В Designer его не всегда видно, можно выставить в коде:

```
cpp
ui->horizontalLayout->setStretch(0, 1); // TabWidget - 1 часть
ui->horizontalLayout->setStretch(1, 3); // правая часть - 3 части
```

Если хотите, чтобы QTabWidget был фиксированной ширины, а правая часть растягивалась — поставьте у QTabWidget Horizontal Policy = Preferred, а у правого виджета Expanding, и задайте minimumWidth / maximumWidth у TabWidget.

Способ 2: вручную в коде (если форма без layout'a)

```
cpp

// 6 конструкторе окна
QWidget *central = new QWidget(this);
QHBoxLayout *layout = new QHBoxLayout(central);

QTabWidget *tabs = new QTabWidget(central);
QTextEdit *right = new QTextEdit(central);

layout->addWidget(tabs);
layout->addWidget(right);
layout->setStretch(0, 1);
layout->setStretch(1, 2);

setCentralWidget(central); // если это QMainWindow
```

Частая ошибка

Если после нажатия «Lay Out Horizontally» ничего не произошло или рамка не появилась — значит, вы не выделили саму форму, а какой-то виджет. В Qt Designer layout всегда ставится на контейнер, а не на отдельный виджет. Убедитесь, что в Object Inspector выделен корень (например, MainWindow), а не tabwidget .

После того как layout поставлен, в Object Inspector у формы появится узел вида horizontalLayout , а у виджетов — свойство layoutStretch и т.д.

Если что-то не получается — скажите, у вас QMainWindow или QWidget в качестве корня формы, и я подскажу точнее.

а больше никак нельзя? что-то оба способа не вызывают доверия

Thought for 1 second

User is skeptical about both approaches. Let me think about what else could work. They have Qt 4.8.6 with Designer. They want a TabWidget on the left that resizes with the window.

Other options:

1. QSplitter - draggable divider

2. QDockWidget - dockable panel (only for QMainWindow)

3. Manual resizeEvent override

4. anchor-like behavior via eventFilter

5. Grid layout instead of horizontal

Let me address their distrust and offer alternatives.

Почему способы вызывают недоверие — и что ещё есть

Понимаю скепсис. Оба способа, которые я привёл, — это стандартный путь, и именно так это делается в 95% проектов на Qt. Но раз хочется альтернатив — вот они, от «правильных» к «экзотическим».

1. QSplitter — если нужен перетаскиваемый разделитель

Часто «прикреплённая слева панель» — это именно QSplitter . Пользователь сам двигает границу, а при растягивании окна пропорции сохраняются автоматически.

В Designer:

1. Киньте на форму `QSplitter` (`Horizontal Splitter`).
2. Внутрь положите `QTabWidget` и правый виджет.
3. Выделите форму → **Lay Out** (любой) — чтобы splitter растягивался вместе с окном.

`QSplitter` сам является контейнером с `layout`'ом внутри, поэтому дополнительных плясок не надо.

2. `QDockWidget` — «настоящая» прикреплённая панель

Если корень формы — `QMainWindow`, то `QDockWidget` создан ровно для этого. Он:

- прилипает к любой стороне (`LeftDockWidgetArea`),
- может сворачиваться/разворачиваться/плавать,
- сам подстраивается под размер окна.

В Designer: `QMainWindow` → в меню **View** → **Dock Widgets** или просто перетащить `QDockWidget` из палитры. Внутрь положить `QTabWidget`.

Это единственный виджет в Qt, который «сам привязывается» без ручного `layout`'а — потому что у `QMainWindow` уже есть встроенный механизм `dock`-областей.

3. `Grid Layout` вместо `Horizontal`

Иногда «не доверяю `HBox`» = «не работает как надо». `QGridLayout` даёт больше контроля: можно задать `columnStretch`, `rowStretch`, `columnMinimumWidth` прямо в Designer.

Выделить форму → **Lay Out in a Grid**. Затем в свойствах формы появится `layoutColumnStretch` вида 1,3 — это те самые пропорции.

4. Ручной `resizeEvent` — без `layout`'ов вообще

Если совсем не хочется `layout`'ов (например, по идеологическим причинам или из-за `legacy`-кода), можно переопределить `resizeEvent`:

```
cpp
void MyWindow::resizeEvent(QResizeEvent *e) {
    QMainWindow::resizeEvent(e);
    int w = e->size().width();
    tabWidget->setGeometry(0, 0, w/4, e->size().height());
    rightWidget->setGeometry(w/4, 0, w - w/4, e->size().height());
}
```

Но: это плохая практика. Вы теряете:

- авто-подгонку под `minimumSize` / `sizeHint` дочерних виджетов,
- работу со шрифтами/DPI,
- нормальное поведение при `sizeConstraint`,
- и переписываете то, что Qt уже умеет.

Прибегать к этому стоит только если виджеты не должны быть в `layout`'е (например, оверлеи, кастомная отрисовка).

5. `QWidget::setSizePolicy` + `QSpacerItem` вручную

Можно без layout'a расставлять виджеты через setGeometry + setSizePolicy — но sizePolicy работает только внутри layout'a. Без layout'a он ничего не делает. Так что этот путь ведёт обратно к layout'ам.

Что я бы посоветовал

Ваша цель	Инструмент
Панель слева, пропорции фиксированы	QHBoxLayout на форме (способ 1)
Панель слева, пользователь двигает границу	QSplitter
Панель слева, сворачивается/плавает	QDockWidget на QMainWindow
Полный контроль над пропорциями	QGridLayout

Layout — это не «один из способов», это единственный штатный механизм в Qt. Всё остальное (QSplitter , QDockWidget) — это просто готовые контейнеры, которые внутри себя тоже используют layout. Если «оба способа не вызывают доверия» — скорее всего, что-то конкретное не сработало (не выделилась форма, layout поставился не туда, виджеты «улетели»). Опишите, что именно происходит при попытке — и я подскажу точную причину.

Какой из вариантов хотите попробовать? Могу расписать по шагам именно для Qt 4.8.6 Designer.

можно мне подробнее привести примеры с кодом как QTabWidget прикрепить справа (я опечталась, ранее неправильно писала)

Thought for 1 second

User wants QTabWidget attached to the right side of the window. Let me give detailed code examples with all approaches: layout, QSplitter, QDockWidget, and manual.

Qt 4.8.6 - so no C++11 features guaranteed, but lambdas work with Qt 4.8. Let me write clean Qt 4 style code.

Let me provide full working examples.

QTabWidget справа — примеры с кодом (Qt 4.8.6)

Разберу три подхода по нарастанию «прикреплённости»: обычный layout, QSplitter, QDockWidget. Для каждого — полный компилируемый пример.

Вариант 1. QHBoxLayout — панель справа

Самый простой и предсказуемый. Слева — основной контент, справа — QTabWidget фиксированной ширины.

main.cpp

```
cpp

#include <QApplication>
#include "mainwindow.h"

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
```

```

MainWindow w;
w.resize(900, 600);
w.show();
return app.exec();
}

```

mainwindow.h

```

cpp

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>

class QTabWidget;
class QTextEdit;

class MainWindow : public QMainWindow
{
    Q_OBJECT
public:
    explicit MainWindow(QWidget *parent = 0);

private:
    QTabWidget *m_tabs;
    QTextEdit *m_editor;
};

#endif // MAINWINDOW_H

```

mainwindow.cpp

```

cpp

#include "mainwindow.h"
#include <QTabWidget>
#include <QTextEdit>
#include <QWidget>
#include <QHBoxLayout>
#include <QLabel>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
{
    // --- центральный виджет и его layout ---
    QWidget *central = new QWidget(this);
    QHBoxLayout *layout = new QHBoxLayout(central);
    layout->setContentsMargins(4, 4, 4, 4);
    layout->setSpacing(4);

    // --- левая часть: основной контент, растягивается ---
    m_editor = new QTextEdit(central);
    m_editor->setPlainText("Основная область слева");

    // --- правая часть: QTabWidget ---
    m_tabs = new QTabWidget(central);
    m_tabs->setTabPosition(QTabWidget::North); // вкладки сверху

    QWidget *page1 = new QWidget;
    QVBoxLayout *p1 = new QVBoxLayout(page1);
    p1->addWidget(new QLabel("Содержимое вкладки 1", page1));
}

```

```

QWidget *page2 = new QWidget;
QVBoxLayout *p2 = new QVBoxLayout(page2);
p2->addWidget(new QLabel("Содержимое вкладки 2", page2));

m_tabs->addTab(page1, "Свойства");
m_tabs->addTab(page2, "История");

// --- ключевое: политики растяжения ---
// редактор тянется, вкладки — остаются примерно своей ширины
m_editor->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
m_tabs->setSizePolicy(QSizePolicy::Preferred, QSizePolicy::Expanding);

// ограничим ширину панели вкладок, чтобы она не разрасталась
m_tabs->setMinimumWidth(220);
m_tabs->setMaximumWidth(400);

layout->addWidget(m_editor, /*stretch*/ 1);
layout->addWidget(m_tabs, /*stretch*/ 0);

setCentralWidget(central);
setWindowTitle("Панель вкладок справа");
}

```

Что тут важно понимать:

- `layout->addWidget(w, stretch)` — второй аргумент это доля растяжения. 1 у редактора, 0 у вкладок: редактор забирает всё свободное место, вкладки остаются своей ширины.
- Если хочется **пропорцию** (например 3:1) — поставьте `layout->addWidget(m_editor, 3); layout->addWidget(m_tabs, 1);` и **уберите** `min/max` ширину у вкладок.
- `setSizePolicy` не работает вне `layout`'а. Здесь он и работает, потому что виджеты уже в `QVBoxLayout`.

Вариант 2. QSplitter — пользователь сам двигает границу

Панель справа, но с «ручкой», за которую можно тянуть. Пропорции сохраняются при растягивании окна.

```

cpp

#include "mainwindow.h"
#include <QSplitter>
#include <QTabWidget>
#include <QTextEdit>
#include <QLabel>
#include <QVBoxLayout>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
{
    QSplitter *splitter = new QSplitter(Qt::Horizontal, this);

    // --- левая часть ---
    m_editor = new QTextEdit(splitter);
    m_editor->setPlainText("Основная область слева");

    // --- правая часть ---
    m_tabs = new QTabWidget(splitter);

```

```

m_tabs->addTab(new QLabel("Вкладка 1"), "Свойства");
m_tabs->addTab(new QLabel("Вкладка 2"), "История");

splitter->addWidget(m_editor);
splitter->addWidget(m_tabs);

// стартовые пропорции
splitter->setStretchFactor(0, 3); // редактор
splitter->setStretchFactor(1, 1); // вкладки

// запрет «схлопнуть» панель в ноль
m_tabs->setMinimumWidth(180);
m_editor->setMinimumWidth(200);

setCentralWidget(splitter);
setWindowTitle("QSplitter: вкладки справа");
}

```

Плюсы: пользователь сам регулирует ширину, при растягивании окна пропорции сохраняются автоматически.

Минусы: граница всегда видна (можно спрятать `splitter->setHandleWidth(0)`, но тогда пропадёт и возможность тянуть).

Вариант 3. QDockWidget — «настоящая» пристыкованная панель

Самый «правильный» способ для QMainWindow. Панель можно перетащить в любую сторону, отцепить, свернуть, закрыть — Qt делает это сам, без единой строчки layout-кода.

```

cpp

#include "mainwindow.h"
#include <QDockWidget>
#include <QTabWidget>
#include <QTextEdit>
#include <QLabel>
#include <QMenuBar>
#include <QMenu>

MainWindow::MainWindow(QWidget *parent)
: QMainWindow(parent)
{
    // --- центральный виджет ---
    m_editor = new QTextEdit(this);
    m_editor->setPlainText("Центральная область");
    setCentralWidget(m_editor);

    // --- док-виджет ---
    QDockWidget *dock = new QDockWidget("Инспектор", this);
    dock->setAllowedAreas(Qt::LeftDockWidgetArea | Qt::RightDockWidgetArea);

    m_tabs = new QTabWidget(dock);
    m_tabs->addTab(new QLabel("Вкладка 1"), "Свойства");
    m_tabs->addTab(new QLabel("Вкладка 2"), "История");

    dock->setWidget(m_tabs);

    // --- пристыковать СПРАВА ---
    addDockWidget(Qt::RightDockWidgetArea, dock);

    // опционально: запретить закрытие/плавание
}

```

```
// dock->setFeatures(QDockWidget::DockWidgetMovable
//                      | QDockWidget::DockWidgetFloatable);

// опционально: пункт меню для показа/скрытия панели
QMenu *viewMenu = menuBar()->addMenu("Вид");
viewMenu->addAction(dock->toggleViewAction());

setWindowTitle("QDockWidget: вкладки справа");
}
```

Как пристыковать справа и не дать утащить влево:

```
cpp

dock->setAllowedAreas(Qt::RightDockWidgetArea); // только справа
```

Начальная ширина у QDockWidget задаётся через `resize()` или `setMinimumWidth()` у самого дока, а не у вкладок:

```
cpp

dock->setMinimumWidth(220);
resizeDocks(QList<QDockWidget*>() << dock, QList<int>() << 280, Qt::Horizontal);
```

(метод `resizeDocks` есть начиная с Qt 4.8 — как раз ваш случай).

Вариант 4. QGridLayout — пропорции в числах

Если хочется управлять пропорциями «в цифрах», `grid` даёт `setColumnStretch` :

```
cpp

QWidget *central = new QWidget(this);
QGridLayout *grid = new QGridLayout(central);

m_editor = new QTextEdit(central);
m_tabs   = new QTabWidget(central);
m_tabs->addTab(new QLabel("A"), "A");
m_tabs->addTab(new QLabel("B"), "B");

grid->addWidget(m_editor, 0, 0);
grid->addWidget(m_tabs,   0, 1);

grid->setColumnStretch(0, 3); // левая колонка: 3 части
grid->setColumnStretch(1, 1); // правая колонка: 1 часть

setCentralWidget(central);
```

Как то же самое сделать в Qt Designer 4.8.6

- 1. Создайте `MainWindow` .
- 2. Перетащите `QTextEdit` (слева) и `QTabWidget` (справа).
- 3. Выделите пустое место формы (в `Object Inspector` должен быть выделен `MainWindow` , а не виджет).
- 4. Нажмите **Form** → **Lay Out Horizontally** (`Ctrl+1` в Qt4 Designer).

- 5. В Object Inspector появится `horizontalLayout` . Выделите его.
- 6. В Property Editor у `horizontalLayout` есть свойство `layoutStretch` — впишите `3,1` . Готово.
- 7. Если хочется зафиксировать ширину панели — в свойствах `tabWidget` задайте `minimumSize / maximumSize` по ширине.

Если **layout «не встаёт»** — почти всегда причина одна: в Object Inspector выделен не корень формы, а один из виджетов. Layout ставится на **контейнер**, а не на виджет.

Что выбрать под задачу

Хочу	Инструмент
Просто панель справа фиксированной ширины	<code>QVBoxLayout</code> (вариант 1)
Панель справа, юзер двигает границу	<code>QSplitter</code> (вариант 2)
Панель можно отцепить, свернуть, перетащить	<code>QDockWidget</code> (вариант 3)
Точные пропорции в цифрах	<code>QGridLayout</code> (вариант 4)

Скажите, какая именно цель — «панель всегда справа и не двигается», «панель с регулируемой границей», или «отцепляемая панель» — и я допишу пример именно под неё (в т.ч. как это уложить в Qt Designer без ручного кода).